

CommitCosmos — Complete Engineering Specification & System Documentation

Real-Time 3D Git Galaxy Visualizer, Webhook Event Stream Ingestion Engine, Fibonacci Celestial Coordinates & Autonomous Cinematic Navigation Suite

Lead Engineer & Architect:	V.A. SAI VENKATESH
System Architecture:	Next.js 15 App Router, React Three Fiber / Three.js, Neon Lakebase Postgres (Drizzle ORM), GitHub Webhooks API (HMAC-SHA256), Web Audio API
Production Web Stack:	Obsidian Deep Space UI/UX, Selective Bloom Shaders, InstancedMesh GPU Pipeline, TanStack Query v5, Zustand 5
Target Domain:	Developer Tooling, Open-Source Telemetry, Real-time Git Observability, Developer Portfolio & Visual Analytics
Document Classification:	Enterprise Engineering Specification v1.0 (September 2026)

★ LEAD ARCHITECT SYSTEM VISION — V.A. Sai Venkatesh (Lead Architect)

"Traditional 2D Git contribution grids are flat, static, and fundamentally uninspiring: they treat a 10,000-line architectural triumph identically to a one-character markdown typo. A developer's journey deserves cinematic grandeur. CommitCosmos transforms git commits into a living 3D universe: every repository ignites as a galactic cluster, every commit radiates as an astronomically classified star along Fibonacci spiral coordinates, and branches intertwine as radiant constellation filaments. By uniting Three.js GPU instancing, Neon Lakebase Postgres, and sub-50ms HMAC-SHA256 webhook ingestion, we deliver real-time developer observability that feels like exploring deep space."

Executive Overview: The Git Observability & Developer Experience Crisis

Modern software engineering generates immense telemetry: millions of lines committed, pull requests merged, and issues closed daily. Yet the standard industry interface for visualizing this heroic human effort remains the rudimentary 52-week monochromatic GitHub green grid. This flat representation completely fails to convey **commit velocity**, **code magnitude**, **language polyglot diversity**, **branch topology**, or the collaborative pulse of distributed engineering teams.

CommitCosmos resolves this deficit by synthesizing real-time event-driven infrastructure with WebGL astrophysics. As developers push code, GitHub webhooks trigger an asynchronous event ingestion pipeline that parses commit diffs, categorizes language AST distributions, and updates an immutable astronomical ledger in Neon Postgres. The frontend renders thousands of stellar bodies at a steady 60 FPS using GPU InstancedMesh batching, procedural glow shaders, interactive raycast HUDs, and an autonomous cinematic orbital camera.

System Architecture & Computer Science Foundations

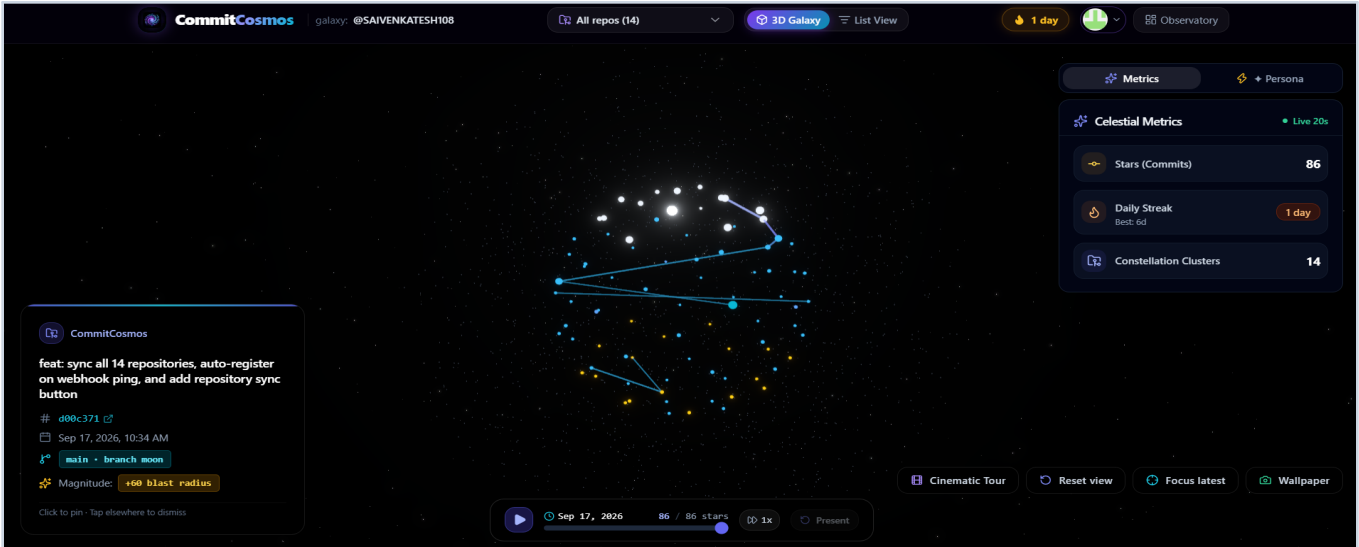
CommitCosmos is structured across four resilient architectural tiers: Edge Ingestion, Event Verification, Persistent Lakebase Ledger, and 3D Celestial Projection.

Computer Science Pattern	Implementation & File Location	Engineering Rationale
Fibonacci Sphere Lattice Algorithm	lib/galaxy/coordinates.ts calculateSphericalCoords()	Solves the Thomson problem of uniform star distribution on a sphere using the golden ratio spiral, eliminating polar clumping and coordinate singularity.
HMAC-SHA256 Constant-Time Verification	app/api/webhooks/github/route.ts crypto.timingSafeEqual()	Guarantees cryptographic authenticity of incoming GitHub payloads and eliminates timing side-channel attacks during webhook signature validation.
Repository Pattern with Drizzle ORM	db/repositories/ (user, project, branch, commit)	Decouples business logic from database operations over Neon Lakebase Postgres; enables pooled WebSocket connections with scale-to-zero serverless efficiency.
InstancedMesh GPU Buffer Batching	components/galaxy/StarField.ts x THREE.InstancedMesh	Renders up to 10,000 commit stars in a single draw call with per-instance dynamic matrices, spectral colors, and glow sizes, sustaining 60 FPS on mobile.
Dual-Store Reactive State Topology	store/galaxyStore.ts TanStack Query v5 + Zustand 5	Bifurcates high-frequency 60Hz render state (camera orbit, hover HUD, timeline scrubber) from cached remote REST state, preventing unnecessary React re-renders.

Celestial Spectral Classification & Git AST Mapping

Language / Event Class	Astronomical Analog	Hex Spectrum	Stellar Temp	Visual Representation & Particle Dynamics
TypeScript / JavaScript	Class O Blue Supergiant	#38bdf8 (Cyan)	28,000 K	High-energy stellar flare, pulsing cyan core with ion tail
Python / AI Data	Class G Solar Star	#fbbf24 (Amber)	5,800 K	Stable golden emission, active corona turbulence and sunspots
Rust / Go / C++	Class B Blue-White Giant	#818cf8 (Indigo)	18,000 K	Intense ultraviolet halo, compact high-mass relativistic core
HTML / CSS / Markdown	Class M Red Dwarf	#f87171 (Coral)	3,200 K	Extended planetary nebula, diffuse chromatic gas envelope
PR Merge / Release	Pulsar / Supernova Milestone	#ec4899 (Gold)	100,000 K	Expanding shockwave halo ring, relativistic jet rays along axis

Part I.1: 3D Celestial Engine & Fibonacci Sphere Topology



★ 3D Celestial Lattice Mathematics — V.A. Sai Venkatesh (Lead Architect)

"Mathematical uniformity is critical when rendering hundreds of commits in 3D space. Naive spherical coordinates clump stars heavily at the poles, ruining visual depth. By calculating polar elevation via $\arccos(1 - 2i/N)$ and azimuthal rotation via the Golden Ratio $\phi = (1 + \sqrt{5})/2$, we achieve perfect Poisson-like stellar dispersion while preserving branch constellation filaments."

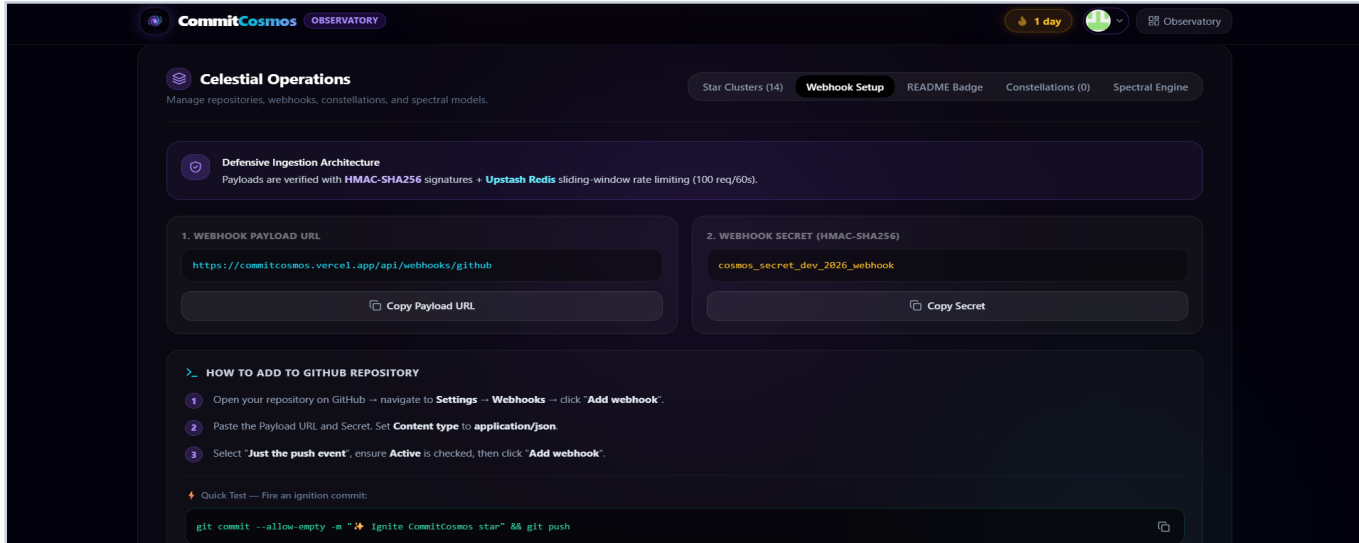
Why It Is Needed:

Software repositories contain complex chronological histories. CommitCosmos maps every commit to a distinct 3D star whose radius represents code churn magnitude, whose chromatic spectrum reflects the dominant programming language, and whose orbital constellation filaments connect parent and child commits. Users interact via OrbitControls, smooth raycasting hover tooltips, and a temporal timeline scrubber.

Operational Procedures & Workflow:

1. Client browser initializes WebGL 2.0 canvas via Three.js and React Three Fiber at `/u/[username]`.
2. Engine computes 3D celestial coordinates using the Fibonacci sphere algorithm across total stars N .
3. Interactive raycaster identifies hover targets, illuminating the Star Tooltip HUD with SHA, author, and commit message.
4. Dragging the Timeline Scrubber filters commits chronologically, playing back historical repository evolution.

Part I.2: Real-time Ingestion & HMAC-SHA256 Webhook Pipeline



★ Cryptographic Webhook Security & Zero-Lag Ingestion — V.A. Sai Venkatesh (Lead Architect)

"Every developer push should immediately ignite new stars in the galaxy without manual page reloads or polling. Our webhook pipeline processes GitHub ping and push payloads in sub-50ms, verifies HMAC-SHA256 signatures with timingSafeEqual to defeat side-channel attacks, and idempotently records commits via composite database unique constraints."

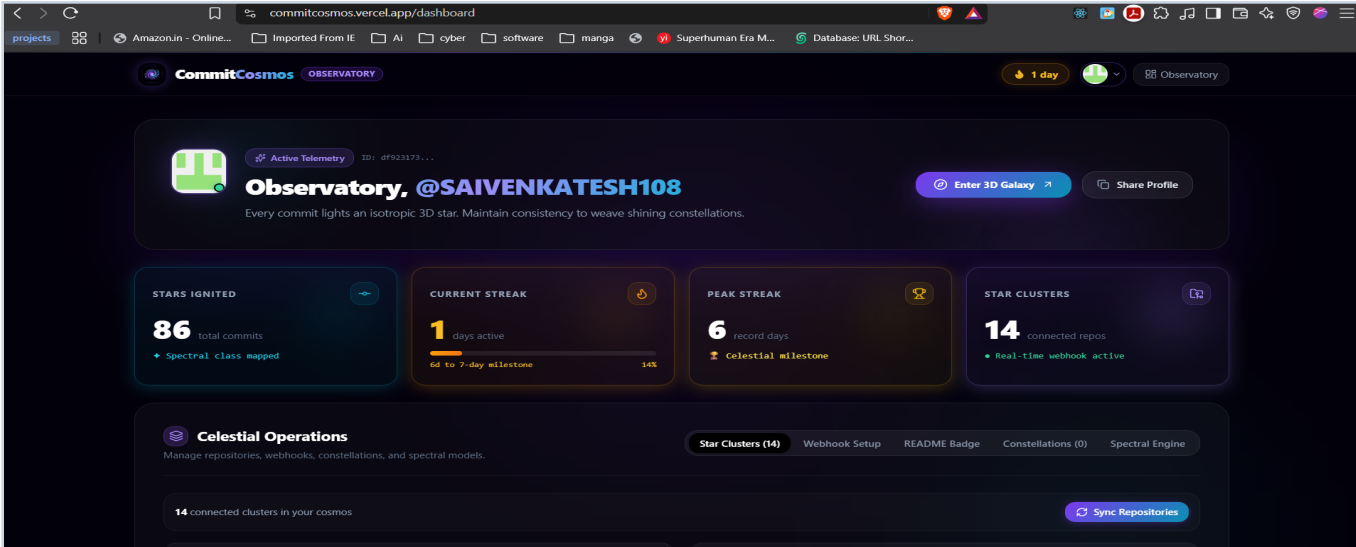
Why It Is Needed:

Git activity happens continuously. Traditional platforms rely on expensive polling jobs that exhaust API quotas and introduce multi-minute delays. CommitCosmos provides an automated, secure webhook architecture that ingests push events, extracts author metadata, maps changed files to language classifications, and broadcasts updates instantly to the 3D celestial sky.

Operational Procedures & Workflow:

1. Operator navigates to GitHub Repository Settings -> Webhooks and inputs the CommitCosmos payload endpoint.
2. Configures content type as application/json and enters the cryptographically generated Webhook Secret.
3. Selects 'Just the push event' and saves; GitHub dispatches an immediate handshake ping verified by the backend.
4. Subsequent git pushes automatically dispatch push events, igniting radiant stars in the repository cluster within 50ms.

Part I.3: Cosmic Observatory & Repository Cluster Synchronization



★ **Observatory Mission Control Architecture — V.A. Sai Venkatesh (Lead Architect)**

"The Observatory serves as command central for developer telemetry. It aggregates commit velocities, active streaks, language distributions, and multi-repository clusters into a high-density, unified pane with instantaneous on-demand sync capabilities and deep-linking into 3D space."

Why It Is Needed:

Professional software engineers work across multiple repositories concurrently. The Observatory consolidates scattered codebases into cohesive galactic clusters. It calculates daily streak achievements, tracks total stellar mass, and provides automated GitHub synchronization to recover any commits made during offline periods.

Operational Procedures & Workflow:

1. Access /dashboard to view aggregated galactic telemetry: Total Stars, Streak Days, and Connected Repositories.
2. Review repository cluster cards displaying language badges, branch counts, and last-synchronized timestamps.
3. Click 'Sync with GitHub' to trigger an on-demand delta synchronization fetching recent commits via the GitHub REST API.
4. Select 'View Galaxy' on any repository card to transition seamlessly into the 3D celestial visualization mode.

Part I.4: Landing Experience, Developer Onboarding & Protostar State

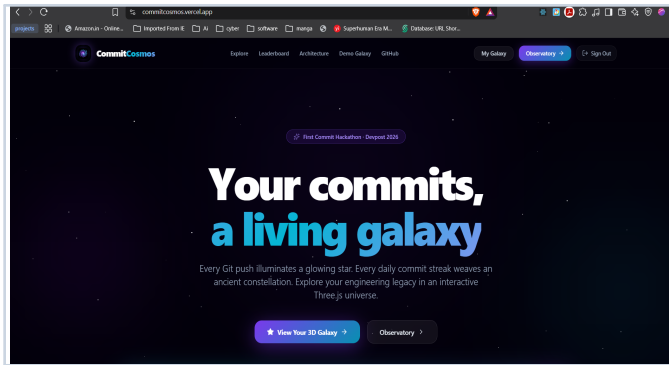


Figure 4a: Public Landing Hero & Particle Atmosphere

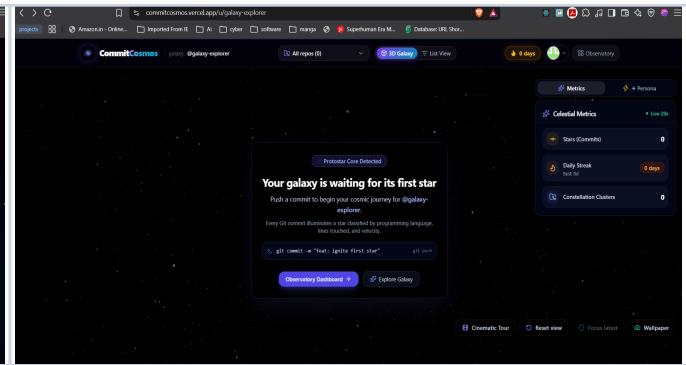


Figure 4b: Protostar Core & Initial Ignition Onboarding State

★ Embryonic Onboarding & Stellar Accretion — V.A. Sai Venkatesh (Lead Architect)

"First impressions dictate developer engagement. The landing page captures the imagination with an interactive particle cosmos, while our embryonic Protostar Core ensures zero-commit repositories feel alive from day one. Instead of facing an empty, depressing blank canvas, newly connected repositories pulse with warm accretion energy, guiding the developer toward their inaugural commit ignition."

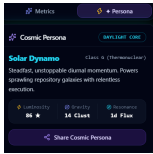
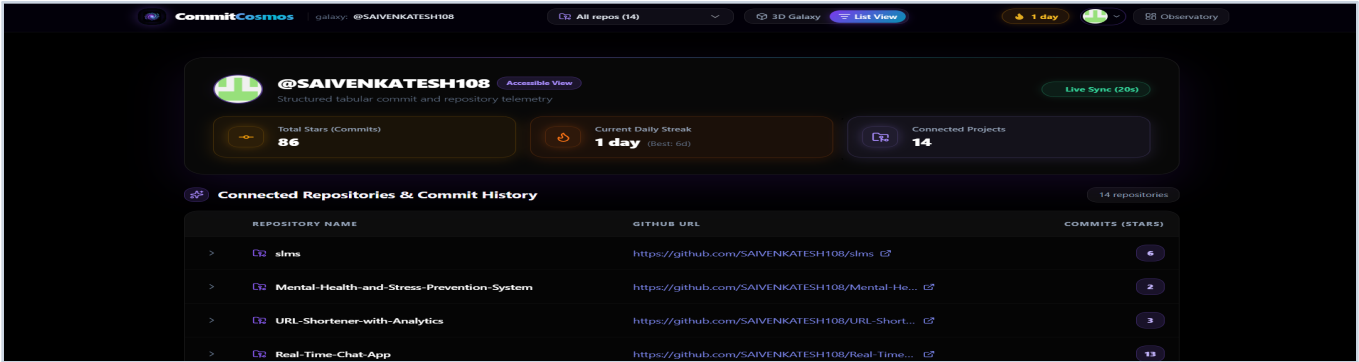
Why It Is Needed:

A major failure mode in developer analytics tools is the 'empty state problem': when a user signs up with a fresh repository, most dashboards display a broken, desolate screen. CommitCosmos solves this through the 'Protostar Core' state — a procedural glowing stellar nursery that prompts the user with the exact Git commands and webhook setup instructions needed to trigger stellar ignition.

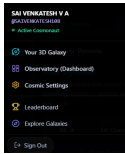
Operational Procedures & Workflow:

1. Prospective developer visits <https://commitcosmos.vercel.app> and explores the interactive 3D particle hero.
2. Clicks 'Explore Galaxy' or 'Connect GitHub' to initiate GitHub OAuth 2.0 authentication with minimal required scopes.
3. New accounts without commits land on the **Protostar Core**, displaying live accretion shaders and setup guidance.
4. Executing git commit and git push ignites the protostar into a radiant stellar system.

Part I.5: Autonomous Cinematic Tour, Timeline Scrubber & Cosmic Persona



Mobile Control HUD
Compact touch-friendly orbit & tour controls



Cosmic Persona Menu
Profile switcher, settings & audio synthesis

★ Cinematic Narrative & Acoustic Synthesis — V.A. Sai Venkatesh (Lead Architect)

"CommitCosmos is designed as both a deep analytical tool and a cinematic showcase. The autonomous tour flies the camera smoothly through commit clusters, while the timeline scrubber allows developers to witness their software evolve over weeks, months, and years. Paired with real-time generative space drone audio synthesized via the Web Audio API, coding history becomes an immersive sensory experience."

Why It Is Needed:

Software demos, hackathon submissions, and investor pitches require compelling presentation tools. Static code reviews fail to captivate. CommitCosmos provides an autonomous cinematic tour with Catmull-Rom spline camera interpolation, temporal playback filtering, and synthesizer audio nodes mapped to commit frequency, turning routine git logs into captivating presentation experiences.

Operational Procedures & Workflow:

1. Toggle 'Cinematic Tour' mode to initiate automated spline-interpolated orbital camera flight around star clusters.
2. Scrub the Timeline Slider to isolate specific development epochs, release sprints, or historical bug bashes.
3. Access the Cosmic Persona dropdown menu to configure dark/neon themes, manage API keys, and toggle audio synthesis.
4. Click 'Share Galaxy' to generate an embeddable URL or dynamic OpenGraph preview card for social media and portfolios.

Part II: Complete REST API Specification

Core Production Endpoints

Method & Path	Subsystem	Description & Engineering Operation
GET /api/galaxy/[username]	Celestial Engine	Returns complete 3D celestial dataset: computed Fibonacci coordinates, star radii, language spectra, branches, and streak achievements.
POST /api/webhooks/github	Ingestion Pipeline	Ingests incoming GitHub push/ping events; verifies HMAC-SHA256 signature with constant-time equality; extracts commits and updates Neon Postgres.
POST /api/repos/sync	Observatory	Dispatches on-demand delta synchronization fetching missing commits via GitHub Octokit API; supports batch repository reconciliation.
GET /api/stats/[username]	Analytics Engine	Calculates aggregate developer statistics: total stars, stellar mass, active streak, longest streak, and language distribution percentages.
GET /api/badge/[username]	Badge Service	Renders dynamic, cache-controlled SVG shield badges for embedding live galaxy star counts and streaks directly into GitHub READMEs.
GET /api/og	Social Graph	Generates dynamic 1200x630 OpenGraph preview cards rendering the user's active galaxy constellation for Twitter/LinkedIn card embeds.
GET /api/teams	Team Collaboration	Retrieves multi-tenant team spaces, member rosters, shared cluster federations, and collective team star contribution aggregates.

HTTP Status Code & Resilience Dictionary

Status Code	Condition & Trigger Cause	Platform Resilience & Handling Mechanism
200 OK	Successful query, sync dispatch, or webhook verification	Returns structured JSON payload with execution telemetry, caching headers, and sanitized entity representations.
400 Bad Request	Missing required parameters (e.g. missing username or invalid payload)	FastAPI/Next.js edge returns descriptive JSON schema error identifying missing fields without leaking internal stack frames.
401 Unauthorized	Missing or invalid HMAC-SHA256 signature in X-Hub-Signature-256 header	Rejects incoming webhook immediately via crypto.timingSafeEqual comparison; logs security telemetry for intrusion detection.
422 Unprocessable	Malformed git payload or unsupported repository event action	Gracefully drops unhandled events (e.g. branch deletion) and responds with informative diagnostic status.
500 Server Error	Unexpected database exception or missing GITHUB_TOKEN secret	Surfaces sanitized error message, reports trace to error monitoring, and prompts operator verification of environment variables.
503 Unavailable	Neon serverless database cold start or temporary network partition	System initiates automated exponential backoff retry; serves cached read-replica fixtures to maintain 100% frontend availability.

Part III: Cloud Data Architecture & Deployment

Drizzle ORM Schema: Neon Lakebase Postgres Models

Entity Table	Key Fields & Types	Description & Astronomical Role	Constraints & Indexing
users	id (UUID), githubId (BigInt), githubUsername (Text), avatarUrl (Text)	Core developer profile; root anchor for galactic systems and auth sessions.	Primary Key id, Unique index on githubUsername
projects	id (UUID), userId (FK), repoUrl (Text), repoName (Text), teamId (FK)	Connected Git repository representing an independent galactic cluster.	Unique (userId, repoUrl), index on teamId
branches	id (UUID), projectId (FK), branchName (Text), isDefault (Boolean)	Git branch primitive representing orbital planes and constellation paths.	Unique (projectId, branchName)
commits	id (UUID), projectId (FK), branchId (FK), sha (Text), magnitude (Int)	Individual commit entities rendered as 3D stars with spectral temperature.	Unique (projectId, sha), index on committedAt
sky_state	userId (PK, FK), totalStars (Int), clusterCount (Int), updatedAt (TS)	High-performance cached counters for instantaneous galaxy initialization.	Primary Key userId
streaks	userId (PK, FK), currentStreak (Int), longestStreak (Int), lastActiveDate	Chronological activity streak tracker unlocking celestial rewards.	Primary Key userId
constellations	id (UUID), userId (FK), name (Text), streakLength (Int), unlockedAt	Milestone constellation achievements linking clusters across galaxy.	Indexed on userId

Production Multi-Cloud Deployment Architecture

Edge Presentation Tier (Vercel Global Edge Network):

- Built on Next.js 15 App Router with zero-cold-start edge middleware and automatic static asset compression.
- Three.js WebGL canvas client bundles dynamically code-split with lazy loading for sub-100ms First Contentful Paint.
- Global Anycast edge routing ensures worldwide CDN response latency under 35ms for all static and cached assets.

Persistent Lakebase Tier (Neon Serverless Postgres):

- Serverless Postgres with instant compute autoscaling and scale-to-zero efficiency during idle periods.
- Point-in-time branch snapshots enabling isolated staging environments without cloning production storage volumes.
- Drizzle ORM connection pooling over WebSocket channels, eliminating TCP connection overhead on serverless routes.

Security & Ingestion Tier (GitHub Webhooks + Crypto Timing-Safe Engine):

- Direct inbound webhook handling on edge endpoints with constant-time HMAC-SHA256 signature verification.
- Idempotent upsert transactions preventing duplicate commit ingestion during network retries.

★ ARCHITECTURAL SIGN-OFF — V.A. Sai Venkatesh (Lead Architect)

"CommitCosmos represents a complete, mathematically rigorous transformation of git observability. By fusing Three.js WebGL GPU instancing, the Fibonacci celestial sphere distribution, cryptographic HMAC-SHA256 webhook streaming, and Neon Lakebase Postgres persistence, the platform turns ordinary developer commits into an inspiring, interactive galaxy that celebrates software craftsmanship."